

Python Built-in Functions

Companion reference for the Real Python "Python Built-in Functions: A Complete Guide" tutorial.
Visit realpython.com to learn more.

Math

`abs(x)`	Absolute value (works on int, float, complex)
`divmod(a, b)`	Returns `(a // b, a % b)` in one call
`max(*args, key=)`	Largest of args or items in iterable
`min(*args, key=)`	Smallest of args or items in iterable
`pow(b, e[, mod])`	b^e; with `mod`, modular exponent
`round(n[, ndigits])`	Banker's rounding to `ndigits`
`sum(it[, start=0])`	Add up an iterable of numbers

```
divmod(17, 5)  # (3, 2)
pow(2, 10, 1000)  # 24
max(["apple","fig"], key=len)  # 'apple'
```

Numbers & Base Conversion

`int(x[, base])`	Parse to int; `base` 2-36 for strings
`float(x)`	Parse to float; accepts `"inf"`, `"nan"`
`complex(r[, i])`	Build complex number `r + i j`
`bool(x)`	Truthiness test (returns `True` / `False`)
`bin(n)`	Int → binary `"0b…"` string
`oct(n)`	Int → octal `"0o…"` string
`hex(n)`	Int → hexadecimal `"0x…"` string
`ord(c)`	1-char string → Unicode code point
`chr(n)`	Code point → 1-char string

```
int("ff", 16)  # 255
bin(10), hex(255)  # ('0b1010', '0xff')
ord("A"), chr(0x1F40D)  # (65, ' ')
```

Strings, Bytes & Format

`str(obj)`	Human-readable string (calls `__str__`)
`repr(obj)`	Unambiguous, often eval-able representation
`ascii(obj)`	Like `repr()` but escapes non-ASCII
`bytes(src[, enc])`	Immutable byte sequence
`bytearray(src)`	Mutable byte sequence
`format(value, spec)`	Apply a format spec (same as f-string)

```
str(3.14)        # '3.14'
repr("hi\n")     # "'hi\\n'"
format(0.125, ".2%")  # '12.50%'
bytes("café", "utf-8")  # b'caf\xc3\xa9'
```

str vs repr: `print()` uses `str()`; the REPL uses `repr()`. Implement `__repr__` on every class.

Collections

`list(it)`	Mutable, ordered sequence
`tuple(it)`	Immutable, ordered sequence
`dict(**kw / pairs)`	Key→value mapping (insertion-ordered)
`set(it)`	Mutable unordered collection of unique items
`frozenset(it)`	Immutable, hashable set (usable as dict key)

```
list(range(3))           # [0, 1, 2]
dict(a=1, b=2)           # {'a': 1, 'b': 2}
dict(zip("ab", [1, 2]))  # {'a': 1, 'b': 2}
set("mississippi")       # {'m','i','s','p'}
```

Constructors are classes: these "functions" are actually built-in types. `isinstance(x, list)` works; `type(x) is list` too.

Iterables & Iteration

Inspect & order

`len(c)`	Number of items in a sized container
`reversed(seq)`	Reverse-order iterator over a sequence
`sorted(it, key=, reverse=)`	New sorted `list`
`all(it)`	`True` if every item is truthy (or empty)
`any(it)`	`True` if at least one item is truthy

Build sequences

`range([s,] stop[, step])`	Lazy arithmetic int sequence
`enumerate(it, start=0)`	Pair each item with its index
`zip(*its, strict=False)`	Parallel iteration; raise on length mismatch
`slice(start, stop[, step])`	Reusable slice object for `seq[s]`

Iterator protocol

`iter(obj)`	Get iterator (calls `__iter__`)
`iter(callable, sentinel)`	Call `callable()` until it returns sentinel
`next(it[, default])`	Pull next value; `default` avoids `StopIteration`
`aiter(obj) / anext(ait)`	Async equivalents (use with `await`)

Functional

`map(fn, *its)`	Lazy iterator applying `fn` to each item
`filter(fn, it)`	Lazy iterator keeping items where `fn(x)` truthy

```
words = ["fig", "apple", "kiwi"]
sorted(words, key=len)
# ['fig', 'kiwi', 'apple']

list(enumerate(words, start=1))
# [(1,'fig'), (2,'apple'), (3,'kiwi')]
```

```
for i, (k, v) in enumerate(zip(keys, vals)):
    print(i, k, v)

list(filter(str.isalpha, "ab12c"))
# ['a', 'b', 'c']

list(map(str.upper, ["a","b"]))  # ['A','B']
```

```
it = iter([1, 2, 3])
next(it)        # 1
next(it, None)  # 2

# Read until empty line:
for line in iter(input, ""):
    process(line)
```

Input & Output

`input([prompt])`	Read a line from stdin (always returns `str`)
`print(*objs, sep=' ', end='\n', file=, flush=)`	Write to stdout or any text file
`open(path, mode='r', encoding=, newline=)`	Open a file; use as context manager
`format(value, spec)`	Apply a single format spec inline

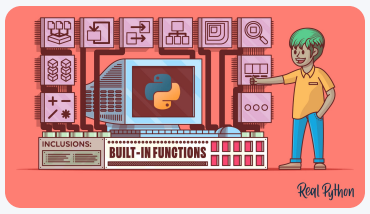
```
name = input("Name: ")
age = int(input("Age: "))

print("a", "b", sep="|", end="!\n")
# a|b!

with open("data.txt", encoding="utf-8") as f:
    for line in f:
        print(line.rstrip())
```

Always pass encoding=: Without it, `open()` uses a platform default and can silently corrupt non-ASCII text. Pair every `open()` with `with`.

All built-ins live in the `builtins` module — already loaded, no import needed.



Python Built-in Functions — continued

Classes & Objects · Scopes & Introspection · Code Execution · Miscellaneous
Visit realpython.com for the full tutorial.

Classes & Objects

Type checks

`type(obj)`	Class of `obj` (exact, no inheritance)
`type(name, bases, dict)`	Dynamically build a new class
`isinstance(obj, cls/tup)`	Honors inheritance & ABCs — preferred
`issubclass(C, base)`	Subclass relationship test
`callable(obj)`	`True` if `obj()` is valid syntax
`object()`	Featureless base of every class

Attribute access

`getattr(obj, name[, default])`	Read attribute by string name
`setattr(obj, name, value)`	Write attribute by string name
`delattr(obj, name)`	Delete attribute by string name
`hasattr(obj, name)`	`True` if access succeeds without error

Inheritance

`super()`	Proxy to next class in the MRO
`super(C, obj)`	Explicit form (rare; outside method body)

Class building blocks

`property(fget, fset, fdel, doc)`	Managed attribute via getter/setter
`classmethod(fn)`	Method receiving `cls`; alt constructors
`staticmethod(fn)`	Plain function inside class namespace

```
class Circle:
    def __init__(self, r):
        self._r = r

    @property
    def area(self):
        return 3.14159 * self._r ** 2

    @classmethod
    def unit(cls):
        return cls(1)

Circle.unit().area  # 3.14159
```

```
class Animal:
    def speak(self): return "..."

class Dog(Animal):
    def speak(self):
        return super().speak() + " woof"

Dog().speak()    # '... woof'
isinstance(Dog(), Animal)  # True
```

```
class Bag:
    pass

b = Bag()
setattr(b, "items", [])
getattr(b, "items").append(1)
hasattr(b, "missing")  # False

callable(len), callable(42)
# (True, False)
```

Prefer isinstance over type: `isinstance()` respects inheritance and accepts a tuple of classes — `isinstance(x, (int, float))`. Reach for `type()` only when you need the exact class.

Scopes & Introspection

`locals()`	Dict of names in the current local scope
`globals()`	Dict of names in the current module's global scope
`id(obj)`	Unique integer id (CPython: memory address)
`dir([obj])`	List names defined on `obj` (or current scope)
`vars([obj])`	`obj.__dict__`, or `locals()` if no arg

```
x = 42
"x" in locals()      # True

globals()["DEBUG"] = True   # write OK at module level

dir(str)[:4]   # ['__add__','__class__', ...]
vars(obj) is obj.__dict__   # True
```

Heads-up: Mutating the dict returned by `locals()` inside a function has undefined effect. Use `globals()` for module-level writes; pass values explicitly otherwise.

Running Code From Strings

`eval(expr[, globals[, locals]])`	Evaluate a single expression, return its value
`exec(code[, globals[, locals]])`	Execute statements; returns `None`
`compile(src, filename, mode)`	Pre-compile to a code object (`'eval'` / `'exec'`)

```
eval("2 ** 10")     # 1024

ns = {}
exec("def square(x): return x*x", ns)
ns["square"](7)     # 49

code = compile("x + 1", "<in>", "eval")
eval(code, {"x": 41})   # 42
```

Security: Never `eval()` / `exec()` untrusted input — it can execute any Python. For data, prefer `ast.literal_eval()` or `json.loads()`.

Miscellaneous

`help([obj])`	Open interactive help; with arg, show its docs
`hash(obj)`	Hash value used by dicts and sets
`__import__(name, …)`	Low-level form of the `import` statement
`memoryview(buffer)`	Zero-copy view over a bytes-like buffer
`breakpoint(*args, **kw)`	Drop into the debugger at this line

```
hash("python")     # stable per process
hash([1, 2])       # TypeError: unhashable

buf = bytearray(b"hello world")
mv = memoryview(buf)[6:]
mv[0] = ord("W")
bytes(buf)         # b'hello World'

def slow(): breakpoint(); return work()
```

breakpoint() tip: Honors `PYTHONBREAKPOINT`. Set it to `0` to disable in production, or `ipdb.set_trace` to swap debuggers without editing code.

At a Glance — Common Idioms

```
sum(x*x for x in range(10))    # 285
", ".join(map(str, [1, 2, 3]))  # '1, 2, 3'
sorted(d.items(), key=lambda kv: kv[1])
max(items, key=attrgetter("score"))
```

```
any(s.startswith("error") for s in lines)
all(0 <= n < 100 for n in scores)
list(zip(*matrix))             # transpose
{c: words.count(c) for c in set(words)}
```

```
with open(path, encoding="utf-8") as f:
    for i, line in enumerate(f, 1):
        print(f"{i:>4} {line.rstrip()}")
```

Shadowed a built-in? Get it back via `import builtins; builtins.list`.